

Lecture 3

EVERYTHING YOU DIDN'T WANT TO KNOW ABOUT
LM ARCHITECTURE AND HYPERPARAMETERS

CS336

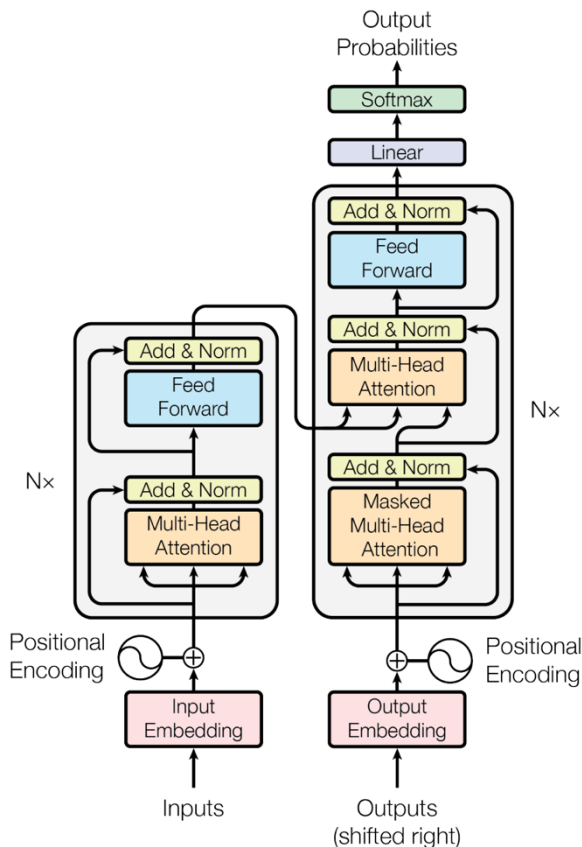
Tatsu H

Outline and goals

- ❖ Quick recap of a modern transformer (what you implement)
- ❖ What do most of the large LMs have in common?
- ❖ What are common variations to the architecture / training process?

Today's theme: the best way to learn is hands-on experience
the second best way is to try to learn from others' experience

Starting point: the ‘original’ transformer



Review: choices in the standard transformer

Position embedding: sines and cosines

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{\text{model}}})$$

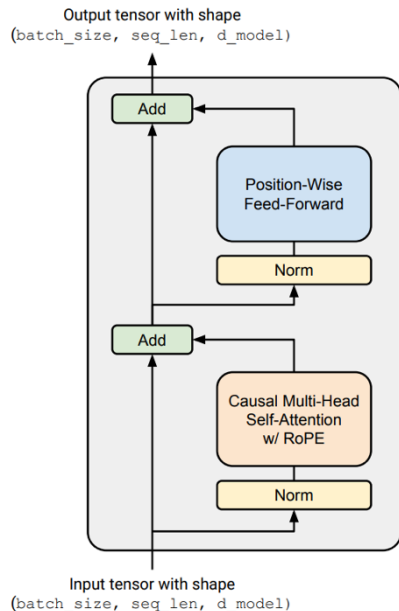
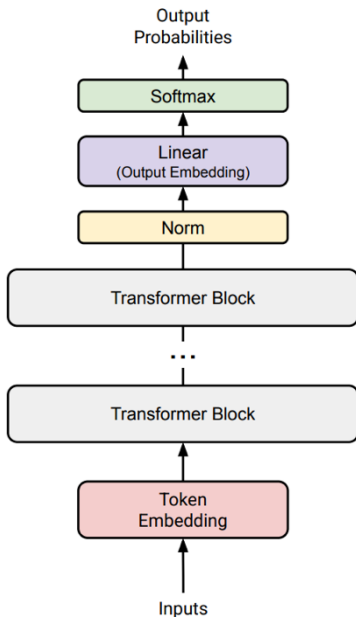
$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{\text{model}}})$$

FFN: ReLU

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

Norm type: post-norm, LayerNorm

What you implemented – simple, modern variant



Differences:

- **LayerNorm** is in front of the block
- **Rotary position embeddings (RoPE)**
- FF layers use **SwiGLU**, not ReLU
- Linear layers (and layernorm) have **no bias** (constant) terms

Why did we pick these?

What should you pick?

ures?

Lc

The Llama 3 Herd of Models

¹A detailed contributor list can be found in the appendix of this paper.

Nemotron-4 340B Technical Report

NVIDIA

FALCON2-11B TECHNICAL REPORT

Coauthors' Names	Number of Publications
Ruxandra Cojocaru	Mugariya Farooq
Sanath Narayan	Ankit Singh
Iohammed Al-Yafeai	Hamza Alobaidi
Kirill Fedyanin	Reda Alami

Abu Dhabi, United Arab Emirates

[do]

M

Technical Report

Harkirat Behl

Sébastien Bubeck

Reka Core, Flash, and Edge: A Series of Powerful Multimodal Language Models

Aitor Ormazabal Che Zheng Cyprien de Masson d'Autume Dani Yogatama

Devu Fu Donovan Ong Eric Chen Eugenie Lamprecht Hai Pham Isaac Ong

Kaloyan Aleksiev Lei Li Matthew Henderson Max Bain Mikel Artetxe

Nishant Relan Piotr Padlewski Qi Liu Ren Chen Samuel Phua

Over 19 new *dense* model releases, many of them with minor architecture tweaks..

InternLM2 Technical Report

[illegible]

Shanghai AI Laboratory
SenseTime Group
The Chinese University of Hong Kong
Fudan University
internlm@pjlab.org.cn

An Yang, Baosong Yan, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyu Lin, Deyi Sheng, Fei Huang, Guanting Dong, Haoran Wei, Huan Lin, Jialong Tan, Jialun Wang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Ma, Jinxian Jing, Jin Xu, Jingren Zhou, Jinze Bai, Jintzheng He, Junyang Lin, Kai Dang, Keming Lu, Keqin Chen, Kexin Yang, Lei Li, Mengfeng Xue, Na Ni, Pei Zhang, Peng Wang, Ru Peng, Rui Mui, Ruiz Ge, Ruiji Lin, Shijie Wang, Shuai Bai, Sinan Tan, Tianshan Zhang, Tianhao Li, Tianyu Li, Tianyu Luo, Wubin Ge, Xiaodong Deng, Xiaohu Zhang, Xiaoji Zhang, Xin Yu, Xinyu Zhang, Xipeng Qiu, Xuancheng Ren, Xuefeng Li, Yang Fan, Yang Yao, Yichang Zhang, Yu Wan, Yufei Chu, Yueqiao Li, Zeyu Chen, Zhenru Zhang, Zhifeng Guo, and Zhihao Fan

Owen Team, Alibaba Group*

Phi-3 Tech
e Language

Microsoft

at a Practical Size

Gemma Team, Google DeepMind¹

f a Small Language Model

ch* Gabriel Martín Blázquez* Guilherme Penedo
ček Agustín Piqueres Lajarin Vaibhav Srivastav
yen Clémentine Fourier Ben Burtenshaw
Cyril Zakka Mathieu Morlon

2024-06-27

d7db9

Page Models

August 5, 2025 Release Product

Introducing gpt-oss

gpt-oss-120b and gpt-oss-20b push the frontier of open-weight reasoning models

[Explore on Hugging Face](#)

[Read model card](#)



DeepSeek-V3.2: Pushing the Frontier of Open Large Language Models

DeepSeek-AI

research@deepseek.com

MiniMax-M1: Scaling Test-Time Efficiently with Lightning Attention

MiniMax¹

Llama 4: Leading Multimodal In

Llama 4 Behemoth

288B active parameters, 18 experts
2T total parameters

The most intelligent teacher model for distillation

[Preview](#)

Llama 4 Maverick

17B active parameters, 128 experts
405B total parameters

Native multimodal with 1M context length

[Preview](#)

Llama 4 Scout

17B active parameters, 18 experts
108B total parameters

Industry leading 10M context length

Optimized inference

[Preview](#)

2025-05-15

IBM Gre
efficient
hybrid models for
enterprise

Published 02 October 2025

Mistral

MiniMax M2 & Agent: Ingenious in Simplicity

[Access API](#)

[Coding Plan](#)

[Try Agent Now](#)



[Try LFM](#) • [Docs](#) • [LEAP](#) • [Disco](#)



NVIDIA Nemotron 3: Efficient and



INTERN-S1: A SCIENTIFIC MULTIMODAL FOUNDATION MODEL

Intern-S1 Team, Shanghai AI Laboratory

ERNIE 4.5 Technical Rep

2025-12-22 · Research

GLM-4.7: Advancing the Coding Capability

[Try it at Z.ai](#) • [Call it at Z.ai](#) • [Z.ai Coding Plan](#) • [GitHub](#) • [HuggingFace](#) • [Tech Report](#)

TECHNICAL REPORT OF KIMI K2

Kimi Team

foundation models
fixture-of-Experts
taining 424B total
MoE architecture,

of Maryland
here.

cost-ef
such a
reason
while S
pre-an

Step-3 is Large yet Affordable: Model-system Co-design for Cost-effective Decoding

StepFun Inc.

models—Nano, Super, and Ultra. These models deliver strong capabilities. The Nemotron 3 family uses a Mixture-of-Experts re to provide best-in-class throughput and context lengths of fcls are trained with NVFP4 and incorporate LatentMoE, a vity. The two larger models also include MTP layers for faster

foundation models—together. We're
ratch, and sharing and programmatically
e, the data, the experiments, the
ite anyone who shares our vision of open
and contribute, whether you want to try out
rithm, dataset, evaluation...there is a lot to

Let's look at the data (on dense architectures)

Learn from the many other models (and papers) out there

Model Name	#	Year	Vocab count	Norm	Paralle Layer	P	Position embed.	Activat.	Other tricks	M.P factor	# param.	# model dim
mT5		2020	250000	RMSNorm	Serial	✓	Relative	GeLU		2.5	24	4096
GPT3 (175B)	OPEN	2020	50257	LayerNorm	Serial	✓	Absolute	GeLU		4	96	12288
GPTJ		2021	50257	LayerNorm	Paralle	✓	RoPE	GeLU		4	28	4096
LaMDA		2021	32000			✓	Relative	GeLU		8	64	8192
Anthropic LM (not claude)		2021	65536			✓				4	64	8192
Gopher (280B)		2021	32000	RMSNorm	Serial	✓	Relative	ReLU		4	80	16384
GPT-neoX		2022	50257	LayerNorm	Paralle	✓	RoPE	GeLU		4	44	6144
BLOOM (175B)		2022	250680	LayerNorm	Serial	✓	Alibi	GeLU		4	70	14336
OPT (175B)		2022	50272	LayerNorm	Serial	✓	Absolute			4	96	12288
PaLM (540B)		2022	256000	RMSNorm	Paralle	✓	RoPE	SwiGLU	Z-loss	4	118	18432
Chinchilla		2022	32000	RMSNorm	Serial	✓	Relative	ReLU		4	80	8192
Baichuan 2		2023	125696	RMSNorm	Serial	✓	Alibi	SwiGLU	Z-loss	2.68	32	4096
Mistral (7B)		2023	32000	RMSNorm	Serial	✓	RoPE	SwiGLU		3.5	32	4096
LLaMA2 (70B)		2023	32000	RMSNorm	Serial	✓	RoPE	SwiGLU		3.5	80	8192
LLaMA (65B)		2023	32000	RMSNorm	Serial	✓	RoPE	SwiGLU		2.6875	80	8192
Olm2		2024	100000	RMSNorm	Serial	✓	RoPE	SwiGLU	Z-loss, QK-norm	2.6875	32	4096
Gemma 2 (27B)		2024	256128	RMSNorm	Serial	✓	RoPE	SwiGLU	Logit soft capping	8	46	4608
Nemotron-4 (340B)		2024	256000	LayerNorm	Serial	✓	RoPE	SwiGLU		4	96	18432
Qwen 2 (72b) - same for 2.5		2024	152064	RMSNorm	Serial	✓	RoPE	SwiGLU		3.609	80	8192
Falcon 2 11B		2024	65024	LayerNorm	Paralle	✓	RoPE	GeLU	Z-loss	4	60	4096
Phi2 (small) - same for phi4		2024	100382	RMSNorm	Serial	✓	RoPE	GeLU		3.5	32	4096
Llama 3 (70B)		2024	128000	RMSNorm	Serial	✓		SwiGLU		3.5	80	8192
Reka Flash		2024	100000	RMSNorm	Serial	✓	RoPE	SwiGLU				
Command R+		2024	256000	LayerNorm	Paralle	✓	RoPE	SwiGLU		2.75	64	12288
OLMo		2024	50304	RMSNorm	Serial	✓	RoPE	SwiGLU		2.6875	32	4096
Qwen (14B)		2024	152064	RMSNorm	Serial	✓	RoPE	SwiGLU		2.675	40	5120
DeepSeek (67B)		2024	100000	RMSNorm	Serial	✓	RoPE	SwiGLU		2.6875	96	8192
Yi (34B)		2024	64000	RMSNorm	Serial	✓	RoPE	SwiGLU		2.857142	60	7168
Marin BB		2025	128256	RMSNorm	Serial	✓	RoPE	SwiGLU		3.5	32	4096
OLMo 3 (7B)		2025	100278	RMSNorm	Serial	✓	Hybrid (SWA+Full)	SwiGLU	QK-norm, Z-loss	2.6875	32	4096
Qwen 3 (8B)		2025	151936	RMSNorm	Serial	✓	RoPE	SwiGLU	QK-norm	3	36	4096
Command A		2025	255000	LayerNorm	Paralle	✓	Hybrid (RoPE+Ne-	SwiGLU				
Gemma 3		2025	262000	RMSNorm	Serial	✓	RoPE	GeLU	Pre-post norm, QK	4	62	5376
SmolLM2 (1.7B)		2025	49152	RMSNorm	Serial	✓	RoPE	SwiGLU		4	24	2048
LLM 2.5 (1.3B)		2026	65536	RMSNorm	Serial	✓	Hybrid (Cores+Full)	SwiGLU	QK-norm	4	16	2048
Gemma 4 E4B (8B)		2026	262144	RMSNorm	Serial	✓	Hybrid (SWA+Full)	GeLU	Logit soft capping	4	42	2060
MiniMax 3 (8B)		2026	131072	RMSNorm	Serial	✓	Hybrid (SWA+Full)	SwiGLU		3.5	34	4096

We will talk through many major architecture and hyperparameter variants.

- What do all these models have in common?
- What parts vary?
- What can we learn from this?

What are we going to cover?

Common architecture variations

- Activations, FFN
- Attention variants
- Position embeddings

Hyperparameters that (do or don't) matter

- What is `ff_dim`? Do `multi_head` dims always sum to `model_dim`?
- How many vocab elements?

Stability tricks

Architecture variations..

Let's think about the core architecture piece

Model Name	Year	# Vocab count	Norm	Parallel Layer	Position embed	Activat.	Other tricks	MLP factor	num. la
mT5	2020	250000	RMSNorm	Serial	Relative	GeLU		2.5	2
GPT2 (175B)	2020	50257	LayerNorm	Serial	Absolute	GeLU		4	9
GPTJ	2021	50257	LayerNorm	Parallel	RoPE	GeLU			2
LaMDA	2021	32000			Relative	GeLU		8	6
Anthropic LM (not claude)	2021	85536						4	6
Gopher (280B)	2021	32000	RMSNorm	Serial	Relative	ReLU		4	8
GPT-NeoX	2022	50257	LayerNorm	Parallel	RoPE	GeLU		4	4
BLOOM (175B)	2022	250480	LayerNorm	Serial	ALiBi	GeLU		4	7
GPT (175B)	2022	50272	LayerNorm	Serial	Absolute	ReLU		4	9
PaLM (540B)	2022	256000	RMSNorm	Parallel	RoPE	SwiGLU	Z-loss	4	11
Chinchilla	2022	32000	RMSNorm	Serial	Relative	ReLU		4	8
BaiChuan 2	2023	125696	RMSNorm	Serial	ALiBi	SwiGLU	Z-loss	2.68	3
Mistral (7B)	2023	32000	RMSNorm	Serial	RoPE	SwiGLU		3.5	3
LLaMA2 (70B)	2023	32000	RMSNorm	Serial	RoPE	SwiGLU		3.5	8
LLaMA (65B)	2023	32000	RMSNorm	Serial	RoPE	SwiGLU	QK-norm	2.6875	8
o1mo 2	2024	100000	RMSNorm	Serial	RoPE	SwiGLU	Z-loss QK-norm	2.6875	3
Gemma 2 (27B)	2024	256128	RMSNorm	Serial	RoPE	GeLU	Logit soft capping Pre-post norm	8	4
Nemotron-4 (340B)	2024	256000	LayerNorm	Serial	RoPE	SwiGLU		4	9
Qwen 2 (72b) - same for 2.5	2024	152064	RMSNorm	Serial	RoPE	SwiGLU		3.609	8
Falcon 2 11B	2024	65024	LayerNorm	Parallel	RoPE	GeLU	Z-loss	4	6
Phi3 (small) - same for phi4	2024	100352	RMSNorm	Serial	RoPE	GeLU		3.5	3
Llama 3 (70B)	2024	128000	RMSNorm	Serial	RoPE	SwiGLU		3.5	8
Reka Flash	2024	100000	RMSNorm	Serial	RoPE	SwiGLU			6
Command R+	2024	256000	LayerNorm	Parallel	RoPE	SwiGLU		2.75	6
o1Mo	2024	50304	RMSNorm	Serial	RoPE	SwiGLU		2.6875	3
Qwen (14B)	2024	152064	RMSNorm	Serial	RoPE	SwiGLU		2.675	4
DeepSeek (67B)	2024	100000	RMSNorm	Serial	RoPE	SwiGLU		2.6875	9
Yi (34B)	2024	64000	RMSNorm	Serial	RoPE	SwiGLU		2.857142	6
Marlin BB	2025	128256	RMSNorm	Serial	RoPE	SwiGLU		3.5	3
o1Mo 3 (7B)	2025	100278	RMSNorm	Serial	Hybrid (SWA+Full)	SwiGLU	QK-norm Z-loss	2.6875	3
Qwen 3 (8B)	2025	151936	RMSNorm	Serial	RoPE	SwiGLU	QK-norm	3	3
Command A	2025	255000	LayerNorm	Parallel	Hybrid (RoPE+Naive)	SwiGLU			6
Gemma 3	2025	262000	RMSNorm	Serial	RoPE	GeLU	Pre-post norm QK-norm	4	6
SmolLM2 (1.7B)	2025	49152	RMSNorm	Serial	RoPE	SwiGLU		4	2
LLM 2.5 (1.2B)	2026	65536	RMSNorm	Serial	Hybrid (Cone+Full)	SwiGLU	QK-norm	4	1
Gemma 4 64B (8B)	2026	262144	RMSNorm	Serial	Hybrid (SWA+Full)	GeLU	Logit soft capping p-Rope QK-norm	4	4
Mistral 3 (8B)	2026	131072	RMSNorm	Serial	Hybrid (SWA+Full)	SwiGLU		3.5	3

High level view:

- Dominance of 'LLaMA-like' architectures
- Trends over the years (QK-norm, Hybrid attention)

Pre-vs-post norm

The one thing *everyone* agrees on (in 2024)

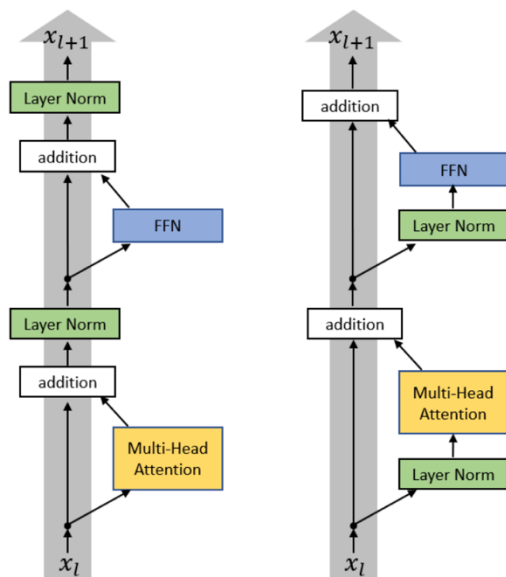


Figure from Xiong 2020

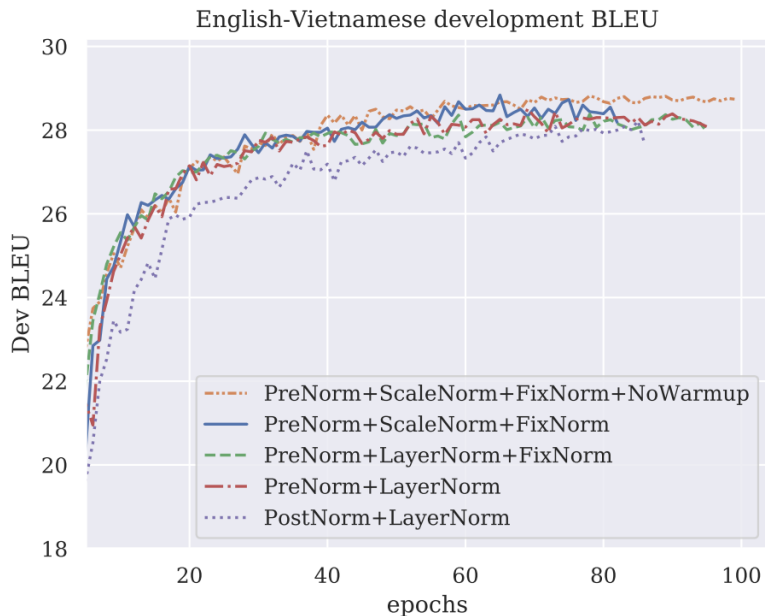
Post-LN Transformer	Pre-LN Transformer
$x_{l,i}^{post,1} = \text{MultiHeadAtt}(x_{l,i}^{post}, [x_{l,1}^{post}, \dots, x_{l,n}^{post}])$	$x_{l,i}^{pre,1} = \text{LayerNorm}(x_{l,i}^{pre})$
$x_{l,i}^{post,2} = x_{l,i}^{post} + x_{l,i}^{post,1}$	$x_{l,i}^{pre,2} = \text{MultiHeadAtt}(x_{l,i}^{pre,1}, [x_{l,1}^{pre,1}, \dots, x_{l,n}^{pre,1}])$
$x_{l,i}^{post,3} = \text{LayerNorm}(x_{l,i}^{post,2})$	$x_{l,i}^{pre,3} = x_{l,i}^{pre} + x_{l,i}^{pre,2}$
$x_{l,i}^{post,4} = \text{ReLU}(x_{l,i}^{post,3}W^{1,l} + b^{1,l})W^{2,l} + b^{2,l}$	$x_{l,i}^{pre,4} = \text{LayerNorm}(x_{l,i}^{pre,3})$
$x_{l,i}^{post,5} = x_{l,i}^{post,3} + x_{l,i}^{post,4}$	$x_{l,i}^{pre,5} = \text{ReLU}(x_{l,i}^{pre,4}W^{1,l} + b^{1,l})W^{2,l} + b^{2,l}$
$x_{l+1,i}^{post} = \text{LayerNorm}(x_{l,i}^{post,5})$	$x_{l+1,i}^{pre} = x_{l,i}^{pre,5} + x_{l,i}^{pre,3}$
Final LayerNorm: $x_{Final,i}^{pre} \leftarrow \text{LayerNorm}(x_{L+1,i}^{pre})$	

Set up LayerNorm so that it doesn't affect the main residual signal path (on the left)

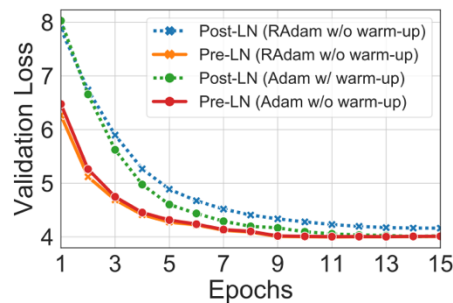
Almost all modern LMs use pre-norm (but BERT was post-norm)

(One somewhat funny exception – OPT350M. I don't know why this is post-norm)

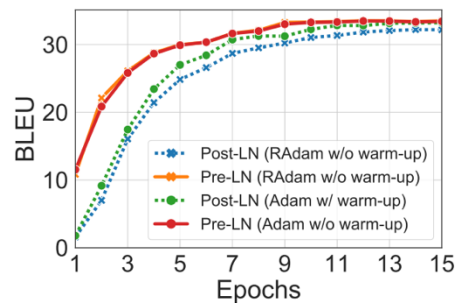
Pre-vs-post-norm, the data



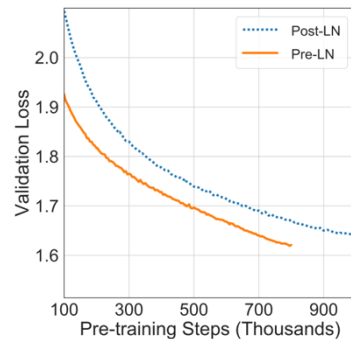
Salazar and Ngyuen 2019



(a) Validation Loss (IWSLT)



(b) BLEU (IWSLT)

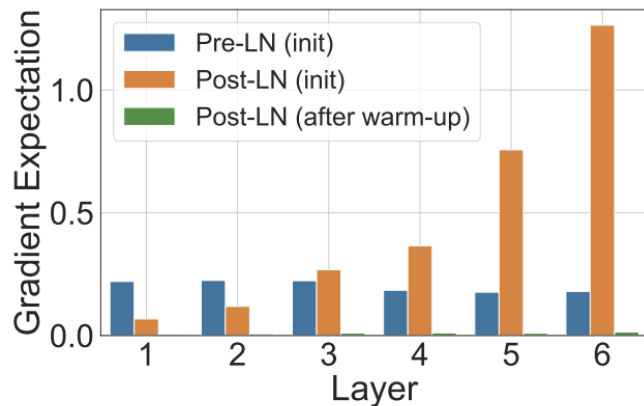


(a) Validation Loss on BERT

Figure from Xiong 2020

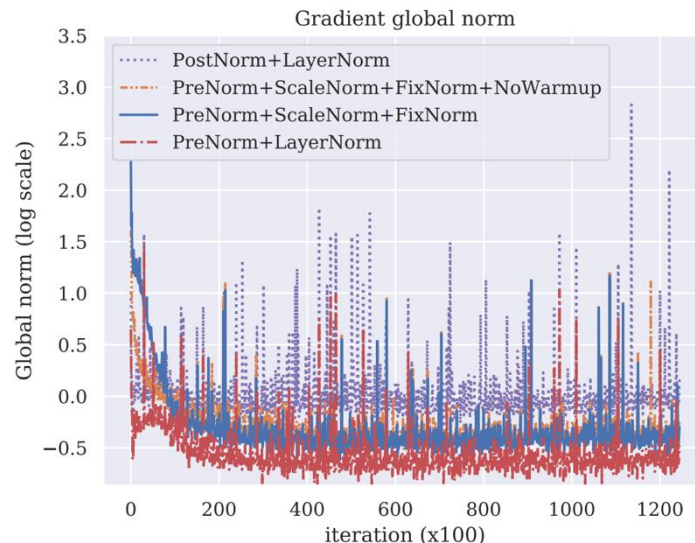
Pre-vs-post norm, explanations?

Gradient attenuation [Xiong 2020]



(a) W^1 in the FFN sub-layers

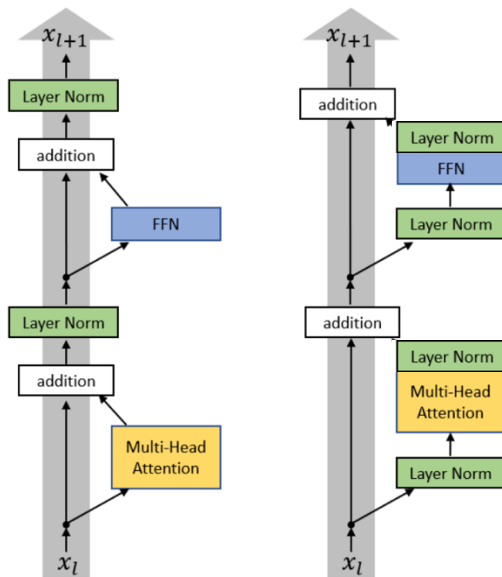
Gradient spikes [Salazar and Ngyuen]



Original stated advantage– removing warmup.
Today – stability and larger LRs for large networks

New things – ‘double’ norm or non-residual postnorm

If putting LayerNorms in residual streams is bad.. Why not post-norm outside the stream?



Recent models: Grok, Gemma 2. Olmo 2 *only* does non-residual post norm

LayerNorm vs RMSNorm

Original transformer: **LayerNorm** – normalizes the mean and variance across d_{model}

$$y = \frac{x - \mathbb{E}[x]}{\sqrt{\text{Var}[x] + \epsilon}} * \gamma + \beta$$

Many modern LMs: **RMSNorm** – does not subtract mean or add a bias term

$$y = \frac{x}{\sqrt{\|x\|_2^2 + \epsilon}} * \gamma$$

Notable models:

GPT3/2/1, OPT, GPT-J, BLOOM

Notable models:

LLaMA-family, PaLM, Chinchilla, T5

Why RMSNorm?

Modern explanation – it's faster (and just as good).

- **Fewer operations** (no mean calculation)
- **Fewer parameters** (no bias term to store)

$$y = \frac{x - \mathbb{E}[x]}{\sqrt{\text{Var}[x] + \epsilon}} * \gamma + \beta$$

Does this explanation make sense?

Operator class	% flop
Δ Tensor contraction	99.80
$\square$ Stat. normalization	0.17
$\circ$ Element-wise	0.03

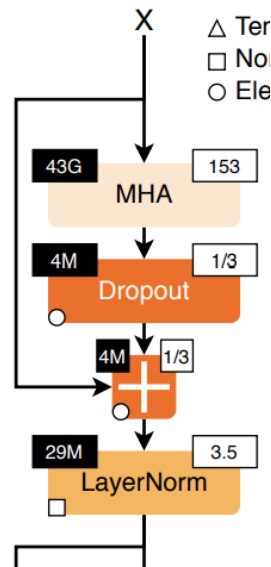
Matrix multiplies are the *vast* majority of FLOPs (and memory)

Why RMSNorm (2)

Important lesson: FLOPS are not runtime! (we will discuss this in far more detail later)

Operator class	% flop	% Runtime
△ Tensor contraction	99.80	61.0
□ Stat. normalization	0.17	25.5
○ Element-wise	0.03	13.5

RMSNorm can still matter due to the importance of *data movement*



Left top ("43G") is FLOPS
Right top ("153") is the FLOP-to-memory ratio

RMSNorm - validation

RMSNorm runtime (and surprisingly, perf) gains have been seen in papers

Model	Params	Ops	Step/s	Early loss	Final loss	SGLUE	XSum	WebQ	WMT EnDe
Vanilla Transformer	223M	11.1T	3.50	2.182 ± 0.005	1.838	71.66	17.78	23.02	26.62
RMS Norm	223M	11.1T	3.68	2.167 ± 0.008	1.821	75.45	17.94	24.07	27.14
Rezero	223M	11.1T	3.51	2.262 ± 0.003	1.939	61.69	15.64	20.90	26.37
Rezero + LayerNorm	223M	11.1T	3.26	2.223 ± 0.006	1.858	70.42	17.58	23.02	26.29
Rezero + RMS Norm	223M	11.1T	3.34	2.221 ± 0.009	1.875	70.33	17.32	23.02	26.19
Fixup	223M	11.1T	2.95	2.382 ± 0.012	2.067	58.56	14.42	23.02	26.31

Narang et al 2020

More generally: dropping bias terms

Most modern transformers don't have bias terms.

Original Transformer:

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

Most implementations (if they're not gated):

$$FFN(x) = \sigma(xW_1)W_2$$

Reasons: memory (similar to RMSnorm) and optimization stability

LayerNorm: recap

- Basically everyone does non-residual norm (often prenorm)
 - Intuition – keep the good parts of residual connections
 - Observations – nicer gradient propagation, fewer spike
 - Some people add a second norm outside the residual stream
- Most people do RMSnorm
 - In practice, works as well as LayerNorm
 - But, has fewer parameters to move around, which saves on wallclock time
 - People more generally drop bias terms since the compute/param tradeoffs are not great.

Activations

A whole zoo of activations ..

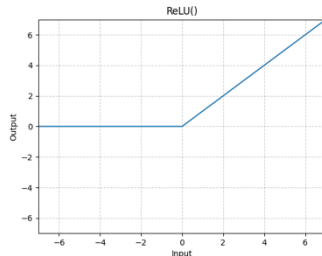
ReLU, GeLU, Swish, ELU, GLU, GeGLU, ReGLU, SeLU, SwiGLU, LiGLU

What are these things? What do people use? Does it matter?

A few of the common activations

ReLU

$$FF(x) = \max(0, xW_1) W_2$$

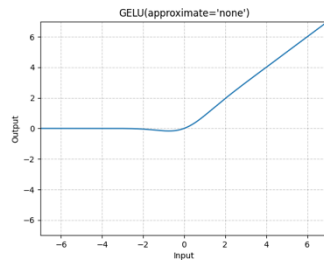


Notable models:

Original transformer, T5, Gopher, Chinchilla, OPT

GeLU

$$FF(x) = \text{GELU}(xW_1)W_2$$
$$\text{GELU}(x) := x\Phi(x)$$



Notable models:

GPT1/2/3, GPTJ, GPT-Neox, BLOOM

SwiGLU / GeGLU (next slide..)

Notable models:

Llama, PaLM, T5 v1.1, *most* models post 2023

Gated activations (*GLU)

GLUs modify the ‘first part’ of a FF layer

$$FF(x) = \max(0, xW_1) W_2$$

Why gate? (my note)

- Gate decides how much; xV carries what (values not clipped at 0)
- Product of two linear maps = quadratic features $\rightarrow$ more expressive per param
- Same params/FLOPs ($d_{ff} = 8/3 \cdot d$); consistent empirical gain, not required (GPT-3 works without it)

Instead of a linear + ReLU, augment the above with an (entrywise) linear term

$$\max(0, xW_1) \rightarrow \max(0, xW_1) \otimes (xV)$$

This gives the gated variant (ReGLU) – note that we have an extra parameter (V)

$$FF_{\text{ReGLU}}(x) = (\max(0, xW_1) \otimes xV) W_2$$

Gated variants of standard FF layers

GeGLU

$$\text{FFN}_{\text{GeGLU}}(x, W, V, W_2) = (\text{GELU}(xW) \otimes xV)W_2$$

Notable models:

T5 v1.1, mT5, LaMDA, Phi3, Gemma 2, Gemma 3, Gemma 4

SwiGLU (swish is $x * \text{sigmoid}(x)$)

$$\text{FFN}_{\text{SwiGLU}}(x, W, V, W_2) = (\text{Swish}_1(xW) \otimes xV)W_2$$

Notable models:

LLaMa 1/2/3, PaLM, Mistral, OlMo, *most models post 2023*

Note: Gated models use smaller dimensions for the d_{ff} by 2/3

Why $\times 2/3$? (my note)

- Std: 2 mats $d \times 4d \rightarrow 8d^2$ params
- GLU: 3 mats (W, V, W_2) $d \times d_{ff}$; $d_{ff} = 8/3 \cdot d \rightarrow 3 \cdot (8/3)d^2 = 8d^2$
- Same params $\rightarrow$ same FLOPs (the $\otimes$ is negligible)
- Cost: gate + value both stored $\rightarrow \sim 5.3d$ vs $4d$ activation mem

Do gated linear units work?

Yes, fairly consistently so.

	Score Average	CoLA MCC	SST-2 Acc
FFN _{ReLU}	83.80	51.32	94.04
FFN _{GELU}	83.86	53.48	94.04
FFN _{Swish}	83.60	49.79	93.69
FFN _{GLU}	84.20	49.16	94.27
FFN _{GEGLU}	84.12	53.65	93.92
FFN _{Bilinear}	83.79	51.02	94.38
FFN _{SwiGLU}	84.36	51.59	93.92
FFN _{ReGLU}	84.67	56.16	94.38
[Raffel et al., 2019]	83.28	53.84	92.68
ibid. stddev.	0.235	1.111	0.569

Do gated linear units work (2)?

Yes, with other works corroborating Shazeer 2020

Model	Params	Ops	Step/s	Early loss	Final loss	SGLUE	XSum	WebQ
Vanilla Transformer	223M	11.1T	3.50	2.182 ± 0.005	1.838	71.66	17.78	23.02
GeLU	223M	11.1T	3.58	2.179 ± 0.003	1.838	75.79	17.86	25.13
Swish	223M	11.1T	3.62	2.186 ± 0.003	1.847	73.77	17.74	24.34
ELU	223M	11.1T	3.56	2.270 ± 0.007	1.932	67.83	16.73	23.02
GLU	223M	11.1T	3.59	2.174 ± 0.003	1.814	74.20	17.42	24.34
GeGLU	223M	11.1T	3.55	2.130 ± 0.006	1.792	75.96	18.27	24.87
ReGLU	223M	11.1T	3.57	2.145 ± 0.004	1.803	76.17	18.36	24.87
SeLU	223M	11.1T	3.55	2.315 ± 0.004	1.948	68.76	16.76	22.75
SwiGLU	223M	11.1T	3.53	2.127 ± 0.003	1.789	76.00	18.20	24.34
LiGLU	223M	11.1T	3.59	2.149 ± 0.005	1.798	75.34	17.97	24.34
Sigmoid	223M	11.1T	3.63	2.291 ± 0.019	1.867	74.31	17.51	23.02
Softplus	223M	11.1T	3.47	2.207 ± 0.011	1.850	72.45	17.65	24.34

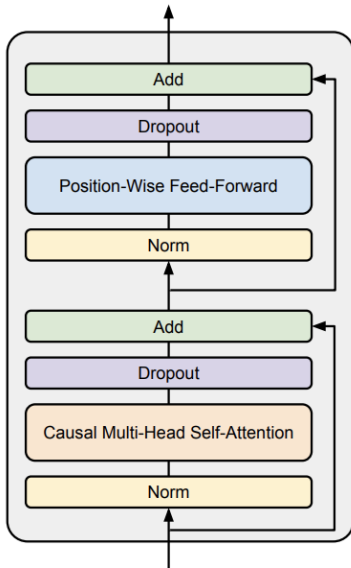
Narang et al 2020

Gating, activations

- **Many variations (ReLU, GeLU, *GLU) across models.**
- ***GLU isn't *necessary* for a working model (see GPT3), but it's rare to see others..**
Some outlier models..
Nemotron 340B (Squared ReLU)
- **Evidence points towards somewhat consistent gains from Swi/GeGLU**

Serial vs Parallel layers

Normal transformer blocks are *serial* – they compute attention, then the MLP



Could we parallelize the transformer block?

Parallel layers

A few models (GPTJ, PaLM, GPT-NeoX) do parallel layers. Originally in GPT-J

Parallel Layers – We use a “parallel” formulation in each Transformer block ([Wang & Komatsuzaki, 2021](#)), rather than the standard “serialized” formulation. Specifically, the standard formulation can be written as:

$$y = x + \text{MLP}(\text{LayerNorm}(x + \text{Attention}(\text{LayerNorm}(x))))$$

Whereas the parallel formulation can be written as:

$$y = x + \text{MLP}(\text{LayerNorm}(x)) + \text{Attention}(\text{LayerNorm}(x))$$

The parallel formulation results in roughly 15% faster training speed at large scales, since the MLP and Attention input matrix multiplications can be fused. Ablation experiments showed a small quality degradation at 8B scale but no quality degradation at 62B scale, so we extrapolated that the effect of parallel layers should be quality neutral at the 540B scale.

If implemented right, LayerNorm can be shared, and matrix multiplies can be fused

Recent Models: Cohere Command A, Falcon 2 11B, Command R+

Summary: architectures

Pre-vs-post norm:

- Everyone does non-residual norm (except OPT350M), likely with good reason.

Layer vs RMSnorm:

- RMSnorm has clear compute wins, sometimes even performance

Gating:

- GLUs are consensus now

Serial vs parallel layers:

- Most models now use serial layers

All Name	#	Year	Norm	Parallel Layer	Pre-norm	Position embedding	Activations
Original transformer		2017	LayerNorm	Serial	☐	Sine	ReLU
GPT		2018	LayerNorm	Serial	☐	Absolute	GeLU
T5 (11B)		2019	RMSNorm	Serial	☒	Relative	ReLU
GPT2		2019	LayerNorm	Serial	☒	Absolute	GeLU
T5 (XXL 11B) v1.1		2020	RMSNorm	Serial	☒	Relative	GeGLU
mT5		2020	RMSNorm	Serial	☒	Relative	GeGLU
GPT3 (175B)		2020	LayerNorm	Serial	☒	Absolute	GeLU
GPTJ		2021	LayerNorm	Parallel	☒	RoPE	GeLU
LaMDA		2021			☒	Relative	GeGLU
Anthropic LM (not claude)		2021			☒		
Gopher (280B)		2021	RMSNorm	Serial	☒	Relative	ReLU
GPT-NeoX		2022	LayerNorm	Parallel	☒	RoPE	GeLU
BLOOM (175B)		2022	LayerNorm	Serial	☒	Alibi	GeLU
OPT (175B)		2022	LayerNorm	Serial	☒	Absolute	ReLU
PaLM (540B)		2022	RMSNorm	Parallel	☒	RoPE	SwiGLU
Chinchilla		2022	RMSNorm	Serial	☒	Relative	ReLU
Mistral (7B)		2023	RMSNorm	Serial	☒	RoPE	SwiGLU
LLaMA2 (70B)		2023	RMSNorm	Serial	☒	RoPE	SwiGLU
LLaMA (65B)		2023	RMSNorm	Serial	☒	RoPE	SwiGLU
GPT4		2023			☐		
Baichuan 2		2023	RMSNorm	Serial	☒	Alibi	SwiGLU
Olmoe 2		2024	RMSNorm	Serial	☐	RoPE	SwiGLU
Gemma 2 (27B)		2024	RMSNorm	Serial	☒	RoPE	GeGLU
Nemotron-4 (340B)		2024	LayerNorm	Serial	☒	RoPE	SqReLU
Qwen 2 (72b) - same for 2.5		2024	RMSNorm	Serial	☒	RoPE	SwiGLU
Falcon 2 11B		2024	LayerNorm	Parallel	☒	RoPE	GeLU
Phi3 (small) - same for phi4		2024	RMSNorm	Serial	☒	RoPE	GeGLU
Llama 3 (70B)		2024	RMSNorm	Serial	☒	RoPE	SwiGLU
Reka Flash		2024	RMSNorm	Serial	☒	RoPE	SwiGLU
Command R+		2024	LayerNorm	Parallel	☒	RoPE	SwiGLU
OLMo		2024	RMSNorm	Serial	☒	RoPE	SwiGLU
Qwen (14B)		2024	RMSNorm	Serial	☒	RoPE	SwiGLU
DeepSeek (67B)		2024	RMSNorm	Serial	☒	RoPE	SwiGLU
Yi (34B)		2024	RMSNorm	Serial	☒	RoPE	SwiGLU
Mixtral of Experts		2024			☐		
Command A		2025	LayerNorm	Parallel	☒	Hybrid (RoPE+NoPE)	SwiGLU
Gemma 3		2025	RMSNorm	Serial	☐	RoPE	GeGLU
SmolLM2 (1.7B)		2025	RMSNorm	Serial	☒	RoPE	SwiGLU

Many variations in position embeddings

Sine embeddings: add sines and cosines that enable localization

$$Embed(x, i) = v_x + PE_{pos}$$

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{model}})$$
$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{model}})$$

Notable models:

Original transformer

Absolute embeddings: add a position vector to the embedding

$$Embed(x, i) = v_x + u_i$$

Notable models:

GPT1/2/3, OPT

Relative embeddings: add a vector to the *attention computation*

$$e_{ij} = \frac{x_i W^Q (x_j W^K + a_{ij}^K)^T}{\sqrt{d_z}}$$

Notable models:

T5, Gopher, Chinchilla

Rope embeddings (next slides..)

Notable models:

GPTJ, PaLM, LLaMA

Most 2024+ models

RoPE: rotary position embeddings

High level thought process: a *relative* position embedding should be some $f(x, i)$ s.t.

$$\langle f(x, i), f(y, j) \rangle = g(x, y, i - j)$$

That is, the attention function *only* gets to depend on the relative position (i-j). How do existing embeddings not fulfill this goal?

- **Sine:** Has various cross-terms that are not relative

$$\langle \text{Embed}(x, i), \text{Embed}(y, i) \rangle = \langle v_x, v_y \rangle + \langle PE_i, v_y \rangle \dots$$

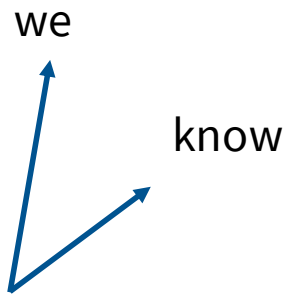
- **Absolute:** obviously not relative

- **Relative embeddings:** $e_{ij} = \frac{x_i W^Q (x_j W^K + a_{ij}^K)^T}{\sqrt{d_z}}$ is not an inner product

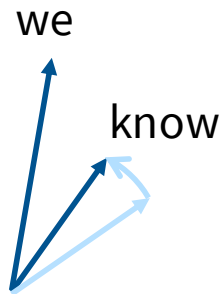
RoPE: rotary position embeddings

How can we solve this problem?

- We want our embeddings to be invariant to absolute position
- We know that inner products are invariant to arbitrary rotation.

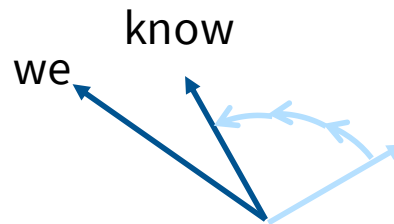


Position independent
embedding



Embedding
“we know that”

Rotate we by ‘0 positions’
know by ‘1 positions’

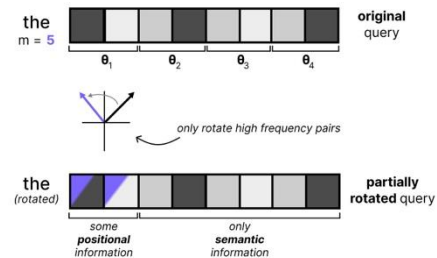
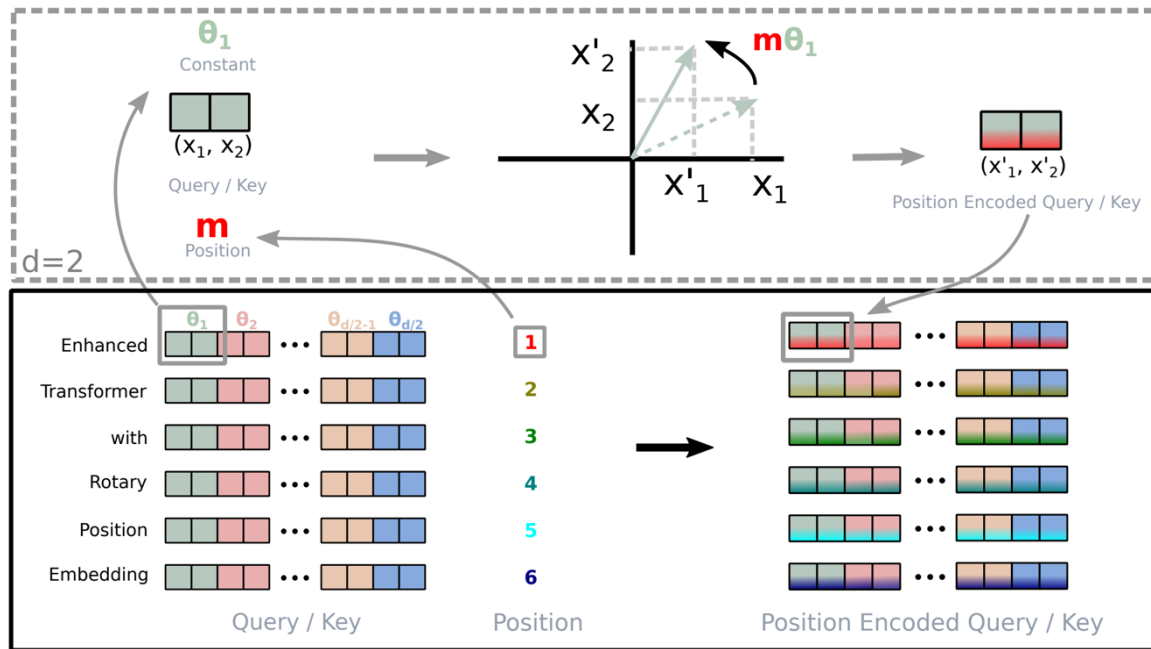


Embedding
“of course we know”

Rotate we by ‘2 positions’
Rotate know by ‘3 positions’

RoPE: rotary position embeddings

There are many rotations, which one do you pick?



Gemma 4 alternative: just first 2

[Su et al 2021]

Just pair up the coordinates and rotate them in 2d (motivation: complex numbers)

The actual RoPE math

Multiply with sines and cosines

$$f_{\{q,k\}}(\mathbf{x}_m, m) = \mathbf{R}_{\Theta, m}^d \mathbf{W}_{\{q,k\}} \mathbf{x}_m \quad (14)$$

$$\mathbf{R}_{\Theta, m}^d = \begin{pmatrix} \cos m\theta_1 & -\sin m\theta_1 & 0 & 0 & \cdots & 0 & 0 \\ \sin m\theta_1 & \cos m\theta_1 & 0 & 0 & \cdots & 0 & 0 \\ 0 & 0 & \cos m\theta_2 & -\sin m\theta_2 & \cdots & 0 & 0 \\ 0 & 0 & \sin m\theta_2 & \cos m\theta_2 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \cdots & \cos m\theta_{d/2} & -\sin m\theta_{d/2} \\ 0 & 0 & 0 & 0 & \cdots & \sin m\theta_{d/2} & \cos m\theta_{d/2} \end{pmatrix} \quad (15)$$

Difference with sine embeddings – not additive, no cross terms

Reading eq. (15) (my note)

- Block-diagonal: $d/2$ independent 2×2 blocks
- Each block = classic 2D rotation $[\cos -\sin; \sin \cos]$
- q split into pairs $(q_1, q_2), (q_3, q_4), \dots$; pair i rotated by angle $m \cdot \theta_i$
- $\theta_i = 10000^{i-1} \cdot (-2(i-1)/d)$: first pairs spin fast, last slow
- $\mathbf{R}(m\theta)^T \mathbf{R}(n\theta) = \mathbf{R}((n-m)\theta)$, so $\langle \mathbf{R}_m \mathbf{q}, \mathbf{R}_n \mathbf{k} \rangle$ depends only on $n-m$

Implementation and code for RoPE

Usual
attention stuff

```
query_states = self.q_proj(hidden_states)
key_states = self.k_proj(hidden_states)
value_states = self.v_proj(hidden_states)

# Flash attention requires the input to have the shape
# batch_size x seq_length x head_dim x hidden_dim
# therefore we just need to keep the original shape
query_states = query_states.view(bsz, q_len, self.num_heads, self.head_dim).transpose(1, 2)
key_states = key_states.view(bsz, q_len, self.num_key_value_heads, self.head_dim).transpose(1, 2)
value_states = value_states.view(bsz, q_len, self.num_key_value_heads, self.head_dim).transpose(1, 2)
```

Get the RoPE
matrix cos/sin

```
cos, sin = self.rotary_emb(value_states, position_ids)
```

Multiply
query/key inputs

```
query_states, key_states = apply_rotary_pos_emb(query_states, key_states, cos, sin)
```

...

Same stuff as the usual multi-head self attention below

Note: embedding at *each attention operation* to enforce position invariance

Hyperparameters

Transformer hyperparameter questions you might have had in 224n..

- How much bigger should the feedforward size be compared to hidden size?
- How many heads, and should num_heads always divide hidden size?
- What should my vocab size be?

And other model setting questions

- Do people even regularize these huge LMs?
- How do people scale these models - very deep or very wide?

Surprising (?) consensus hyperparameter 1

Feedforward – model dimension ratio.

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

There are two dimensions that are relevant – the feedforward dim (d_{ff}) and model dim (d_{model}). What should their relationship be?

$$\mathbf{d}_{ff} = 4 \mathbf{d}_{model}$$

This is *almost always* true. There's just a few exceptions.

Exception #1 – GLU variants

Remember that GLU variants scale down by $2/3^{\text{rd}}$. This means most GLU variants have

$d_{ff} = \frac{8}{3}d_{model}$. This is mostly what happens. Some notable such examples.

Model	d_{ff}/d_{model}
PaLM	4
Mistral 7B	3.5
LLaMA-2 70B	3.5
LLaMA 70B	2.68
Qwen 14B	2.67
DeepSeek 67B	2.68
Yi 34B	2.85
T5 v1.1	2.5

Models are roughly in this range, though PaLM, LLaMA2 and Mistral are slightly larger

Exception #2 – T5

As we have (and will) see, most LMs are have boring, conservative hyperparameters. One exception is T5 [Raffel et al 2020] which has some *very bold* settings.

In particular, for the 11B model, they set

$$\begin{aligned}d_{ff} &= 65,536 \\d_{model} &= 1024\end{aligned}$$

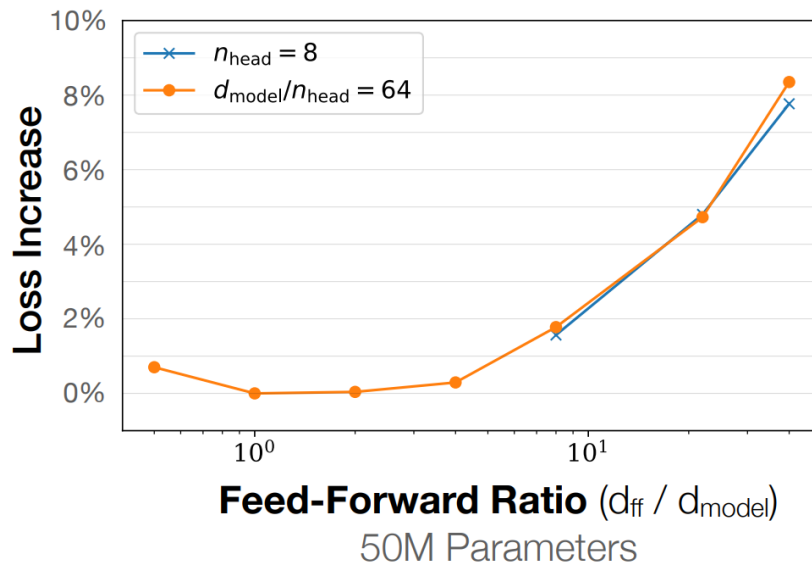
For an astounding 64-times multiplier.

for “11B” we use $d_{ff} = 65,536$ with 128-headed attention producing a model with about 11 billion parameters. We chose to scale up d_{ff} specifically because modern accelerators (such as the TPUs we train our models on) are most efficient for large dense matrix multiplications like those in the Transformer’s feed-forward networks.

Other, recent exceptions – Gemma 2 (8x), SmolLM/Gemma 3/Gemma 4 (4x, GLU)

Why this range of multipliers?

Empirically, there's a basin between 1-10 where this hyperparameter is near-optimal



What can we learn from the model-dim hyperparam?

- The ‘default’ choices of $d_{ff} = 4d_{model}$ and $d_{ff} = 2.66d_{model}$ have worked well for nearly all modern LLMs.
- But T5 does show that even radical choices of $d_{ff} = 64d_{model}$ can work. This hyperparameter choice isn’t written in stone.
- That said, T5 has a follow-up model (T5 v1.1) that is ‘improved’ and uses a much more standard 2.5 multiplier on GeGLU, so the 64-times multiplier is likely suboptimal.

Surprising (?) consensus hyperparameter 2

Head-dim* num-heads to model-dim ratio. As a reminder, slide from 224n.

Multi-head self-attention is computationally efficient

- Even though we compute h many attention heads, it's not really more costly.
 - We compute $XQ \in \mathbb{R}^{n \times d}$, and then reshape to $\mathbb{R}^{n \times h \times d/h}$. (Likewise for XK, XV .)
 - Then we transpose to $\mathbb{R}^{h \times n \times d/h}$; now the head axis is like a batch axis.
 - Almost everything else is identical, and the **matrices are the same sizes**.

This doesn't *have to* be true: we can have head-dimensions $>$ model-dim / num-heads.

Per-head view (my note)

- $Q/K/V$ ($d \times d$) = h blocks of $d \times d/h$, side by side
- One XQ matmul = all heads; reshape splits cols
- QK^T , softmax, $\cdot V$ per head (head = batch axis)
- W_o ($d \times d$) mixes the heads back together

But most models do follow this guideline

How many heads, whats the model dim?

Some examples of this hyperparameter

	Num heads	Head dim	Model dim	Ratio
GPT3	96	128	12288	1
T5	128	128	1024	16
T5 v1.1	64	64	4096	1
LaMDA	128	128	8192	2
PaLM	48	258	18432	1.48
LLaMA2	64	128	8192	1
Qwen 3.5 (27B)	24	256	5120	1.2

Most models have ratios around 1 – notable exceptions by some google models.

Aspect ratios

Should my model be deep or wide? *How* deep and how wide?

Most models are surprisingly consistent on this one too!

Sweet spot?

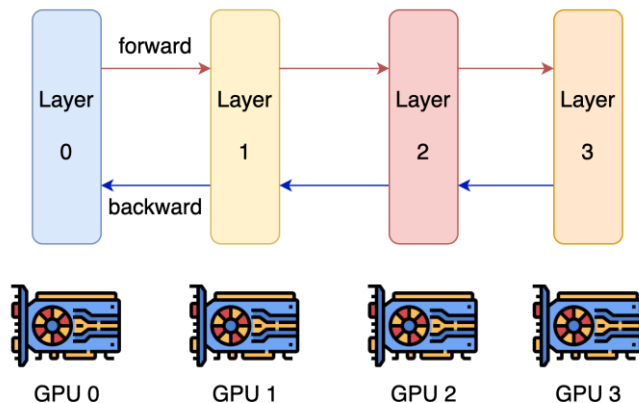
Model	d_{model}/n_{layer}
BLOOM	205
T5 v1.1	171
PaLM (540B)	156
GPT3/OPT/Mistral/Qwen /OLMo 3	128
LLaMA / LLaMA2	102
Gemma 3	87
Gemma 4	61
T5 (11B)	33

Considerations about aspect ratio

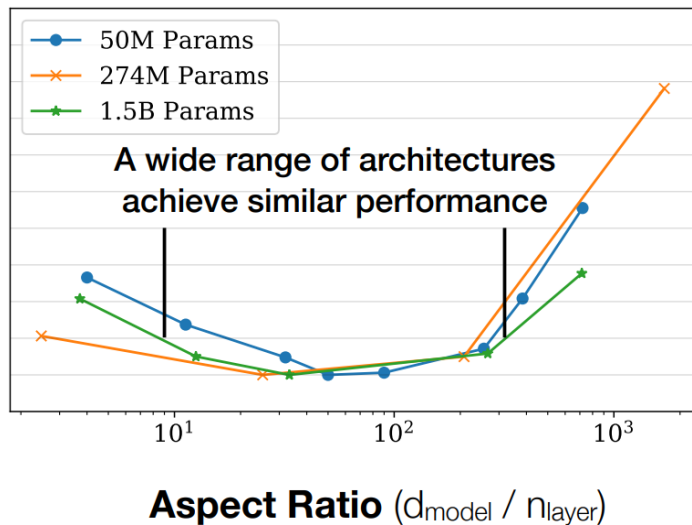
Extremely deep models are harder to parallelize and have higher latency

The Limits of Depth vs Width We note an obvious limitation with our advice. Scaling depth has an obvious limiter, i.e., they are non-parallelizable across different machines or devices and every computation has to always wait for the previous layer. This is unlike width, which can be easily parallelizable over thousands or hundreds of thousands of devices. Within the limitation of scaling

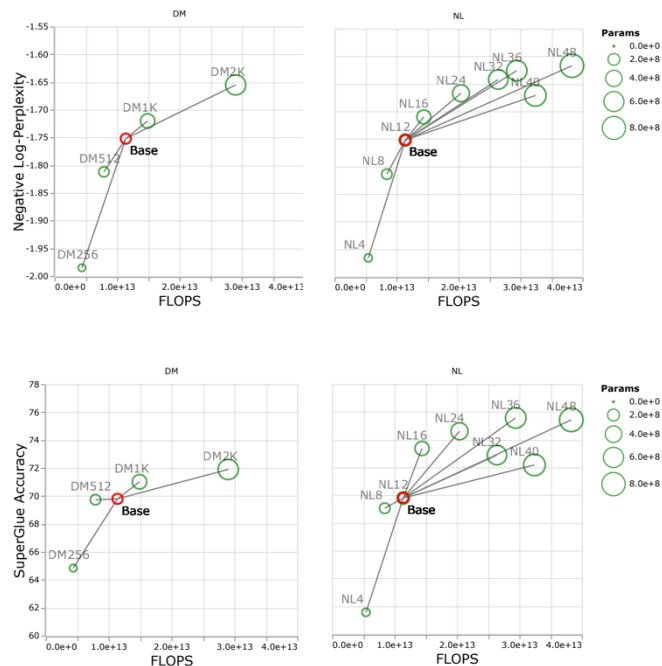
[Tay et al 2021]



Evidence on aspect ratio scaling



[Kaplan et al 2020]



[Tay et al 2021]

What are typical vocabulary sizes?

Monolingual models – 30-50k vocab

Model	Token count
Original transformer	37000
GPT	40257
GPT2/3	50257
T5/T5v1.1	32128
LLaMA	32000

Multilingual / production systems 100-250k

Model	Token count
mT5	250000
PaLM	256000
GPT4	100276
Gemma 4	262144
DeepSeek	100000
Qwen 15B	152064
Yi	64000

Monolingual vocabs don't need to be huge, but multilingual ones do

Dropout and other regularization

Do we need regularization during pretraining?

Arguments against:

- There is *a lot* of data (trillions of tokens), more than parameters.
- SGD only does a single pass on a corpus (hard to memorize)

This is all quite reasonable.. but what do people do in practice?

Dropout and weight decay in practice

Model	Dropout*	Weight decay
Original transformer	0.1	0
GPT2	0.1	0.1
T5	0.1	0
GPT3	0.1	0.1
T5 v1.1	0	0
PaLM	0	(variable)
OPT	0.1	0.1
LLaMA	0	0.1
Qwen 14B	0.1	0.1

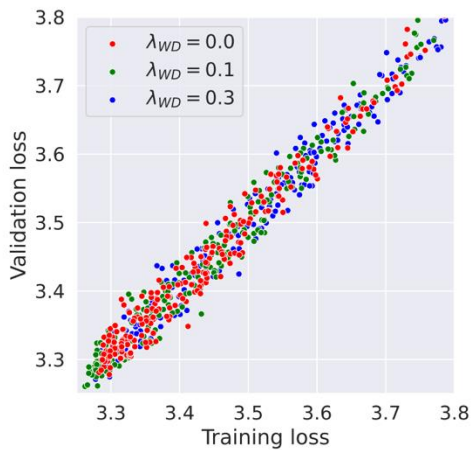
Many older models used dropout during pretraining

Newer models (except Qwen) rely only on weight decay

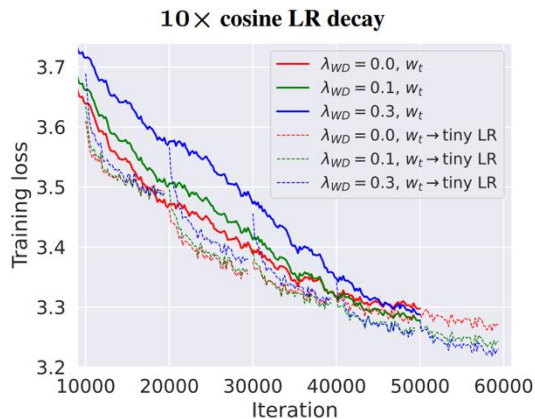
* Most of the times papers just don't discuss dropout. On open models, this closely matches not doing dropout. This may not be true of closed models.

Why weight decay LLMs?

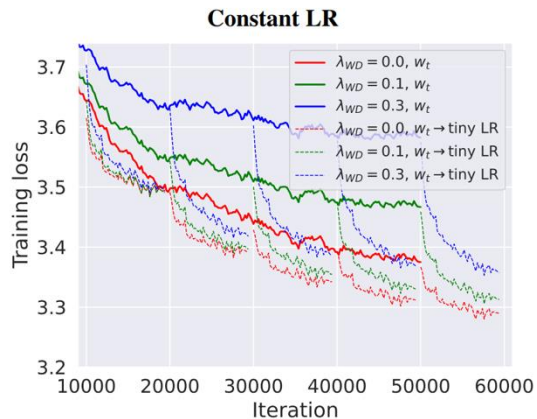
[Andriushchenko et al 2023] has interesting observations about LLM weight decay



It's not to control overfitting



Weight decay interacts with learning rates (cosine schedule)



Summary: hyperparameters

Feedforward

- Factor-of-4 rule of thumb (8/3 for GLUs) is standard (with some evidence)

Head dim

- Head dim * Num head = D model is standard – but low to no validation

Aspect ratio

- Wide range of ‘good’ values (100-200). Systems concerns dictate the value

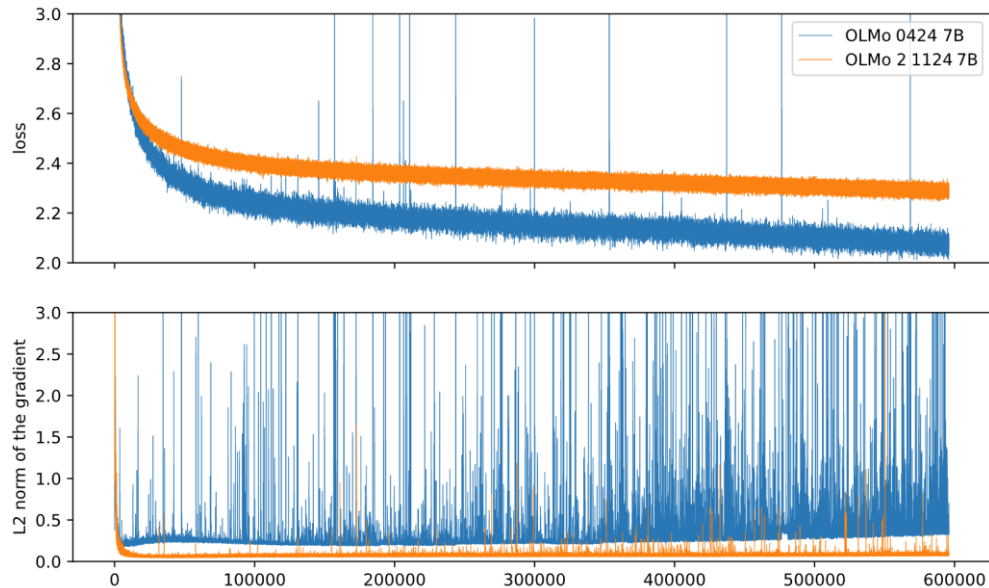
Regularization

- You still ‘regularize’ LMs but its effects are primarily on optimization dynamics

A3 Name	# Year	# MLP factor	Σ Aspect ratio (d/layer)	# weight decay	# drop_rate
Original transformer	2017	4	85	0	0.1
GPT	2018	4	64	0.1	0.1
T5 (11B)	2019	64	43	0	0.1
GPT2	2019	4	33	0.1	0.1
T5 (XXL 11B) v1.1	2020	2.5	171	0	0
mT5	2020	2.5	171	0	0
GPT3 (175B)	2020	4	128	0.1	0.1
GPTJ	2021		146	0.1	0
LaMDA	2021	8	128		
Anthropic LM (not claude)	2021	4	128		
Gopher (280B)	2021	4	205		
GPT-NeoX	2022	4	140	0.01	0
BLOOM (175B)	2022	4	205	0.1	0
OPT (175B)	2022	4	128	0.1	0.1
PaLM (540B)	2022	4	166		0
Chinchilla	2022	4	102		
BaiChuan 2	2023	2.68	128	0.1	0
Mistral (7B)	2023	3.5	128	0.1	0
LLaMA2 (70B)	2023	3.5	102	0.1	0
LLaMA (65B)	2023	2.6875	102	0.1	0
GPT4	2023		0		
Olmo 2	2024	2.6875	128		
Gemma 2 (27B)	2024	8	100		
Nemotron-4 (340B)	2024	4	192		0
Qwen 2 (72b) - same for 2.5	2024	3.609	102		
Falcon 2 11B	2024	4	68	0.1	
Phi3 (small) - same for phi4	2024	3.5	128		
Llama 3 (70B)	2024	3.5	102		0
Reka Flash	2024		0		
Command R+	2024	2.75	192		
OLMo	2024	2.6875	128	0.1	0
Qwen (14B)	2024	2.675	128	0.1	0.1
DeepSeek (67B)	2024	2.6875	86	0.1	0
Yi (34B)	2024	2.857142	119	0.1	0
Mixtral of Experts	2024		0		
Command A	2025		0		
Gemma 3	2025	4	87		
SmolLM2 (1.7B)	2025	4	85		

Stability tricks

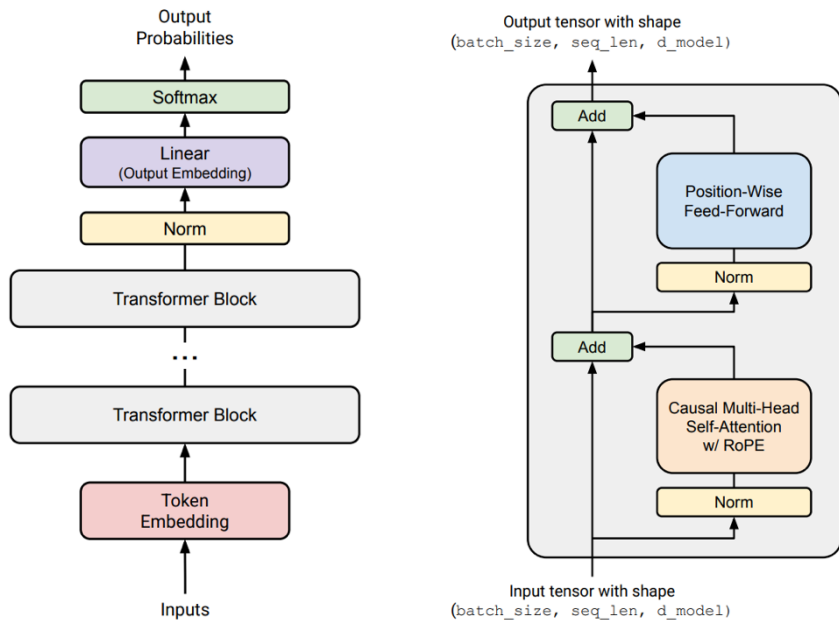
Recently, lots of attention on *stable training*



Don't train models that look like the blue curve!

Where do the issues arise? Beware of softmaxes!

Softmaxes – can be ill-behaved due to exponentials / division by zero



Output softmax stability – the ‘z-loss’

Recall the softmax calculation

$$\begin{aligned}\log(P(x)) &= \log\left(\frac{e^{U_r(x)}}{Z(x)}\right) \\ &= U_r(x) - \log(Z(x)) \\ Z(x) &= \sum_{r'=1}^{|V|} e^{U_{r'}(x)}\end{aligned}$$

$$\begin{aligned}L &= \sum_i [\log(P(x_i)) - \alpha(\log(Z(x_i)) - 0)^2] \\ &= \sum_i [\log(P(x_i)) - \alpha \log^2(Z(x_i))]\end{aligned}$$

[From Devlin 2014]

Not a reweighting of Z (my note)

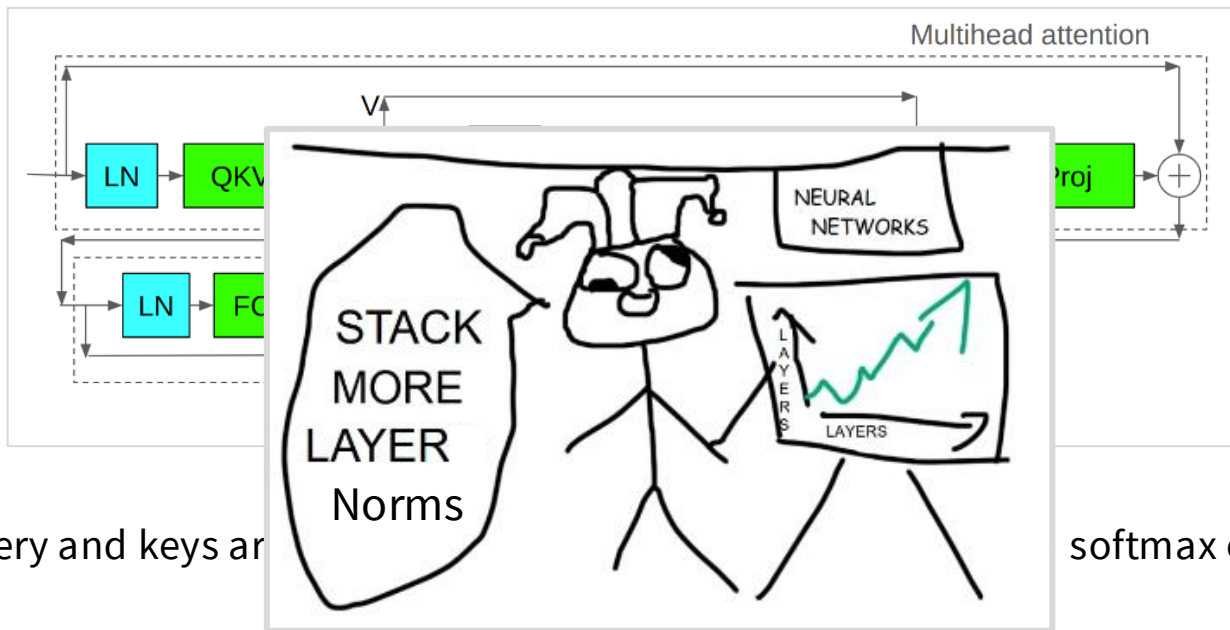
- CE keeps its $\log Z$ (coefficient 1); z-loss ADDS an L2 penalty on $\log Z$ — weight decay on the logit scale, not a smaller Z term.
- CE sees only logit differences, so the level drifts freely; $\alpha \cdot \log^2 Z$ pins it ($Z \approx 1$, $U_r \approx \log P$) via $\text{grad } 2\alpha \cdot \log Z \cdot p_j$ on every logit.

This is useful for stability! PaLM used this ‘z loss’ trick.

We additionally use an auxiliary loss of $z_loss = 10^{-4} \cdot \log^2 Z$ to encourage the softmax normalizer $\log(Z)$ to be close to 0, which we found increases the stability of training.

Other examples: Baichuan 2 (2023), DCLM (2024), OLMo 2 (2025), OLMo 3 (2025)

Attention softmax stability – the ‘QK norm’



Other examples: DCLM, OLMo2, Gemma 2, Qwen3, OLMo 3, Gemma 4

Originally from vision and multimodal models [Dehgani 2023, Idefcs, Chameleon]

Logit soft-capping.

Soft-capping the logits to some maximum value via Tanh

Logit soft-capping. We cap logits ([Bello et al., 2016](#)) in each attention layer and the final layer such that the value of the logits stays between $-\text{soft_cap}$ and $+\text{soft_cap}$. More specifically, we cap the logits with the following function:

$$\text{logits} \leftarrow \text{soft_cap} * \tanh(\text{logits}/\text{soft_cap}).$$

We set the `soft_cap` parameter to 50.0 for the self-attention layers and to 30.0 for the final layer.

Prevents logits from blowing up, but also might have perf issues?

Table 4: Models perplexity with confidence interval ± 0.1 at 95% level.

bf16 baseline	<i>soft_cap</i>	<i>QKV_norm</i>	<i>QK_norm_cap</i>	<i>QK_norm</i>	<i>QK_FC_norm</i>
11.19	11.24	10.85	11.00	10.84	10.87

Attention heads

Most models don't touch the attention heads much at all with a few minor exceptions..

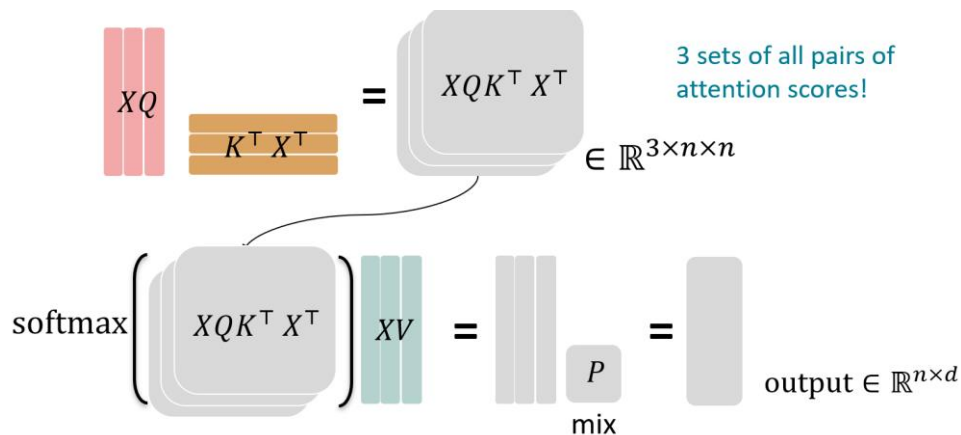
GQA / MQA : Saving inference costs by reducing the number of heads

Sparse or sliding window attention (GPT4/Mistral): restricting the attention pattern to reduce compute cost

Exotic SSM stuff (Jamba, Falcon 3, Qwen 3.5, etc): next lecture!

GQA/MQA – Reducing attention head cost

Let's think about the compute involved for attention



d = hidden dim
b = batch
n = length (<d)
h = heads
k = head dim (d/h)

Total arithmetic operations (bnd^2), **total memory accesses** ($bnd + bhn^2 + d^2$)

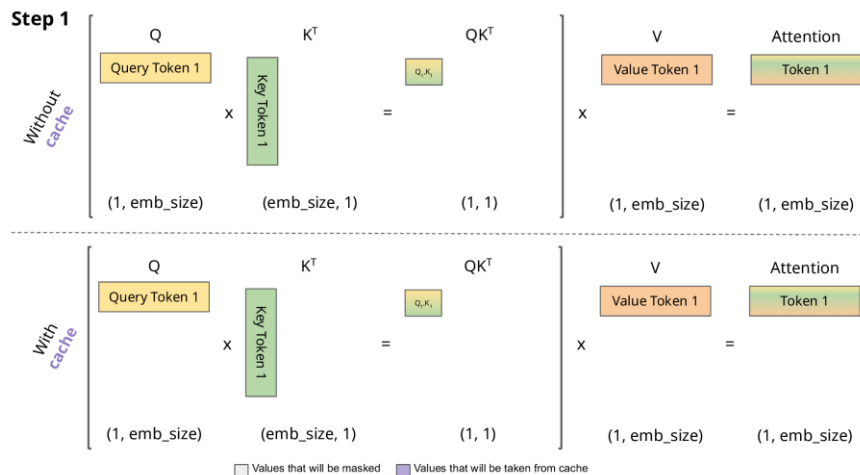
Arithmetic intensity (compute/memory) is high $O\left(\left(\frac{1}{k} + \frac{1}{bn}\right)^{-1}\right)$ - we can keep our GPUs running

GQA/MQA – Reducing attention head cost

What about the *incremental* case when we generate text?

Key difference: can't parallelize the generation process – needs to be step by step

In this case – we need to incrementally re-compute/update attention via the 'KV cache'



GQA/MQA – Reducing attention head cost

What's the incremental arithmetic intensity?

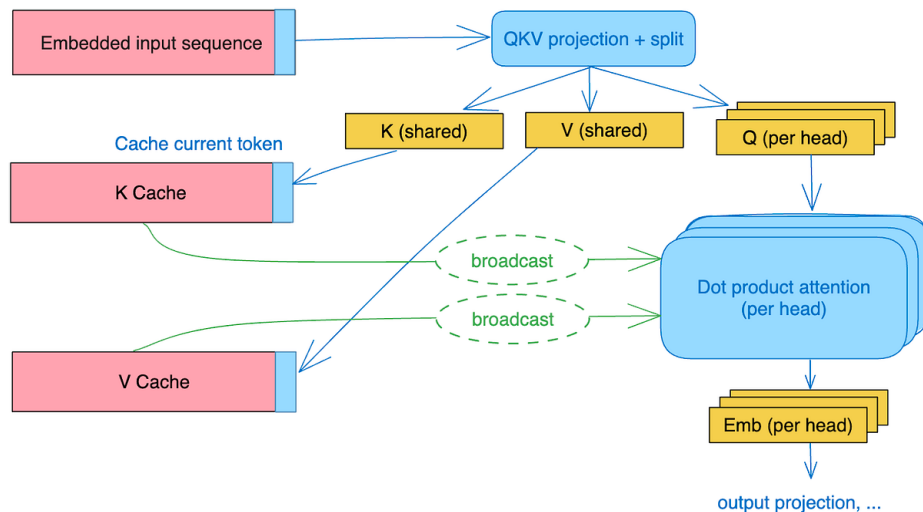
Total arithmetic operations (bnd^2), **total memory accesses** ($bn^2d + nd^2$)^{K, V projection}

Arithmetic intensity is not good $O\left(\left(\frac{n}{d} + \frac{1}{b}\right)^{-1}\right)$ - need large batches + short seq length
(n) or big model dimensions (d)

Is there some way around this? The n/d term is difficult to reduce.

MQA – just have fewer key dimensions.

Key idea – have multiple queries, but just one dimension for keys and values

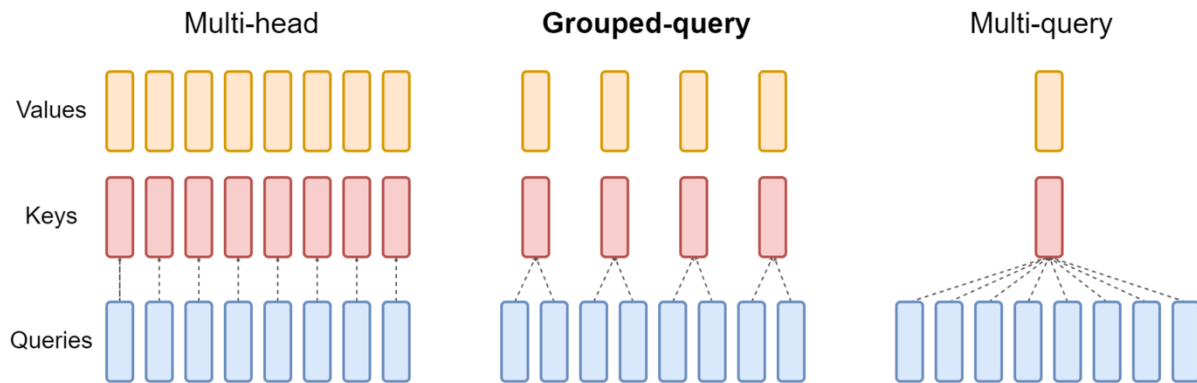


We have much fewer items to move in and out of memory (KV Cache)

Total memory access $(bnd + bn^2k + nd^2)$, **Arithmetic intensity** $O\left(\left(\frac{1}{d} + \frac{n}{dh} + \frac{1}{b}\right)^{-1}\right)$

Additional extensions – GQA

Don't go all the way to one dimension of KV – have fewer dims



Simple knob to control expressiveness (key-query ratio) and inference efficiency

More recently – MLA (multihead latent attention) from deepseek v2

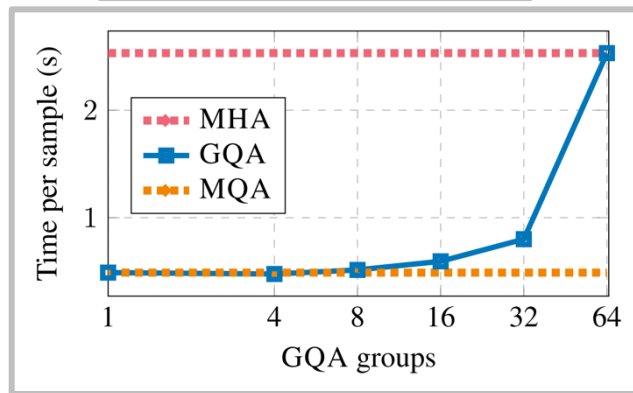
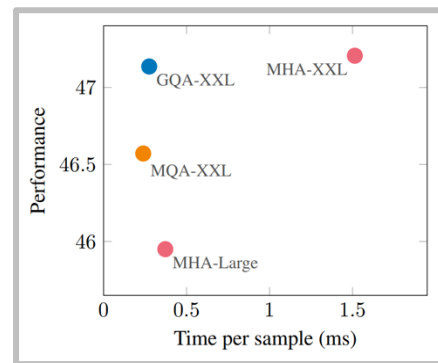
Does MQA hurt? Sometimes..

Small PPL hit w/ MQA [Shazeer 2019]

Table 3: Billion-Word LM Benchmark Results.

Attention	h	d_k, d_v	d_{ff}	dev-PPL
multi-head	8	128	8192	29.9
multi-query	8	128	9088	30.2
multi-head	1	128	9984	31.2
multi-head	2	64	9984	31.1
multi-head	4	32	9984	31.0
multi-head	8	16	9984	30.9

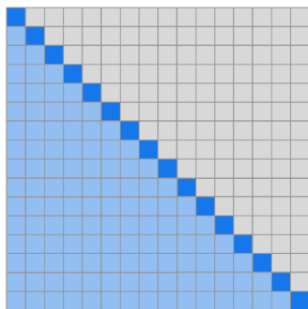
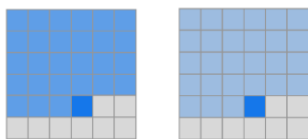
Low/no hit w/ GQA [Ainslie 2023]



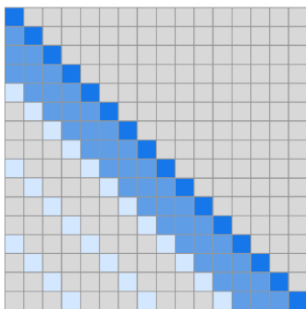
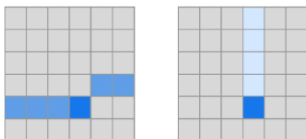
Sparse / sliding window attention

Attending to the entire context can be expensive (quadratic).

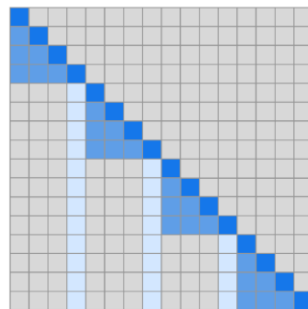
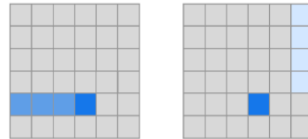
Build sparse / structured attention that trades off expressiveness vs runtime (GPT3, GPT-OSS, Gemma4)



(a) Transformer



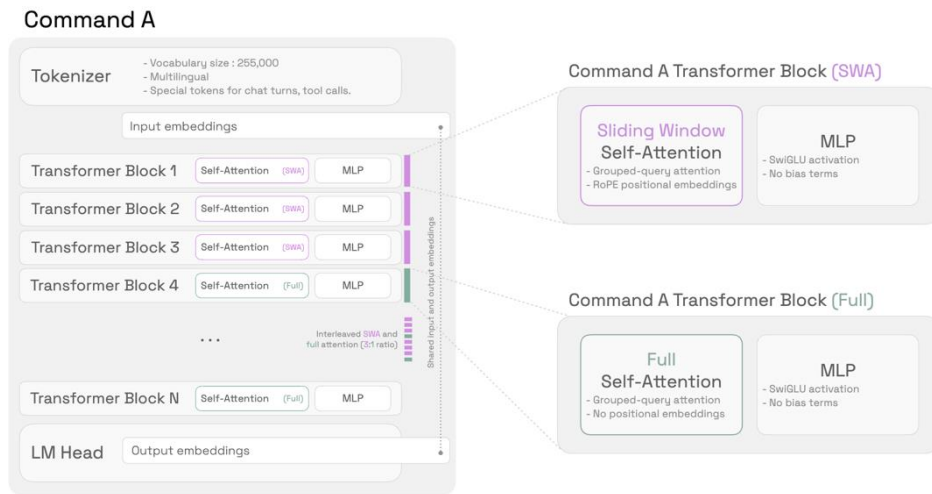
(b) Sparse Transformer (strided)



(c) Sparse Transformer (fixed)

Current standard trick – interleave ‘full’ and ‘LR’ attention

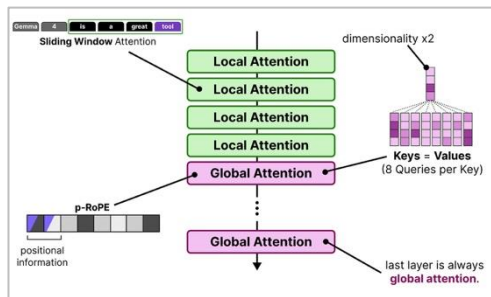
From Cohere Command A – Every 4th layer is a full attention



Long-range info via NoPE, short-range info via RoPE + SWA.

Other models – LLaMA 4, Gemma 3, Gemma 4, OLMo 3 does SWA+Full RoPE.

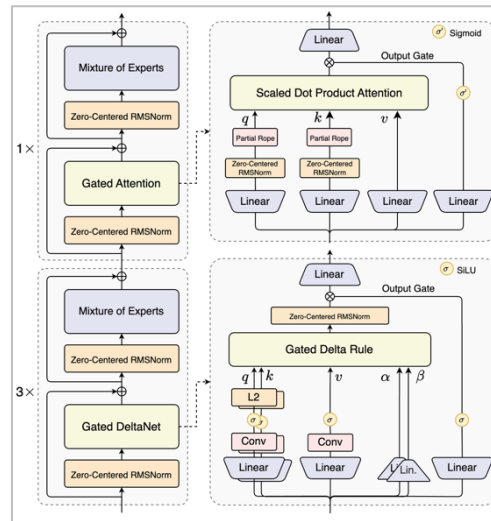
Other recent examples of interleaved attention



Gemma 4

Gradient clipping	1.0
Z-loss weight	10^{-5}
Weight decay on embeddings	No
Sliding window attention	3/4 of layers; 4,096 tokens
RoPE scaling	YaRN on full attn. layers
RoPE θ	$5 \cdot 10^5$
Layer norm applied to	Outputs

Olmo 3



Qwen 3.5 / Qwen 3 Next

Recap, conclusion, etc.

Many aspects (arch, hparams) of transformers are in common across the big LMs

Model Name	Repo	Year	Model Size	Params	Parallel Layer	Pre-norm	Position Embedding	Activations	Mod	Parameterization	Other Tricks	MLP Factor	num_layers	model_dim	Aspect ratio (short)	num_heads	dim_val
Original transformer	huggingface.co/docs/transformers/v4.28.0/en/transformers.html	2017	37000	37000	Serial	□	None	Swish	□			4	6	512	85	8	0.1
GPT	openai.com/research/gpt-2	2018	400M	400M	Serial	□	Absolute	Gelu	□			4	12	768	64	12	0.1
GPT2	openai.com/research/gpt-2	2019	900M	900M	Serial	□	Absolute	Gelu	□			4	48	1600	32	12	0.1
T5 (1.1B)	google.com/papers/p17603	2019	321.2B	321.2B	Serial	■	Relative	Gelu	□			64	24	1024	43	128	0.1
GPT3 (175B)	openai.com/research/gpt-3	2020	900M	900M	Serial	□	Absolute	Gelu	□			4	96	12288	128	96	0.1
StB	huggingface.co/docs/transformers/v4.28.0/en/transformers.html	2020	250000	250000	Serial	■	Relative	Swish	□			2.5	24	4096	171	64	0
T5 (XXL 1.1B v1.1)	google.com/papers/p17603	2020	321.2B	321.2B	Serial	■	Relative	Gelu	□			2.5	24	4096	171	64	0
DeBERTa (240M)	microsoft.com/papers/p17603	2021	32000	32000	Serial	■	Relative	Gelu	□			4	80	16384	205	128	0
DeBERTa (1.1B)	microsoft.com/papers/p17603	2021	65536	65536	Serial	■	Relative	Gelu	□			4	64	8192	128	128	0
DeBERTa (3.0B)	microsoft.com/papers/p17603	2021	32000	32000	Serial	■	Relative	Gelu	□			4	64	8192	128	128	0
GPTJ	huggingface.co/docs/transformers/v4.28.0/en/transformers.html	2021	900M	900M	Parallel	■	None	Gelu	□			4	18	4096	146	16	0
Chinchilla	huggingface.co/docs/transformers/v4.28.0/en/transformers.html	2022	32000	32000	Serial	■	Relative	Gelu	□			4	80	8192	102	64	0
PaLM (540B)	google.com/papers/p17603	2022	250000	250000	Parallel	■	Relative	Gelu	□	2.5x		4	118	16432	166	48	0
GPT (1.1B)	openai.com/research/gpt-3	2022	900M	900M	Serial	□	Absolute	Gelu	□			4	96	12288	128	96	0.1
BLOOM (1.1B)	bigscience.ai	2022	250000	250000	Serial	■	None	Gelu	□			4	70	14336	205	112	0
GPT-NeoX	openai.com/research/gpt-3	2022	900M	900M	Parallel	■	None	Gelu	□			4	64	8192	128	128	0
GPTA	openai.com/research/gpt-3	2023	100000	100000	Serial	□	None	Gelu	■			4	64	8192	128	128	0
LLaMA (65B)	meta.com/papers/p17603	2023	32000	32000	Serial	■	Relative	Gelu	□			2.875	80	8192	102	64	0
LLaMA (70B)	meta.com/papers/p17603	2023	32000	32000	Serial	■	Relative	Gelu	□			3.5	80	8192	102	64	0
Mistral (7B)	mistral.ai	2023	32000	32000	Serial	■	Relative	Gelu	□			3.5	32	4096	128	32	0
Baichuan 2	baichuan.ai	2023	125000	125000	Serial	■	Relative	Gelu	□	2.5x		2.88	32	4096	128	32	0
Mistral of Experts	mistral.ai	2023	65000	65000	Serial	■	Relative	Gelu	□			2.875	80	7168	110	64	0
Yi (34B)	01all.com	2024	100000	100000	Serial	■	Relative	Gelu	□			2.875	80	8192	88	64	0
DeepSeek (67B)	deepseek.com	2024	16384	16384	Serial	■	Relative	Gelu	□			2.875	40	8192	128	40	0.1
Qwen (1.1B)	qwenlm.github.io	2024	100000	100000	Serial	■	Relative	Gelu	□			2.875	32	4096	128	32	0
Qwen (72B)	qwenlm.github.io	2024	250000	250000	Serial	■	Relative	Gelu	□			2.75	64	12288	192	96	0
Reika Flash	reika.ai	2024	100000	100000	Serial	■	Relative	Gelu	□			3.5	80	8192	102	64	0
LLaMA 3 (70B)	meta.com/papers/p17603	2024	128000	128000	Serial	■	Relative	Gelu	□	MLP		3.5	32	4096	128	32	0
Phi3 (small) - same for phi3	microsoft.com/papers/p17603	2024	100000	100000	Serial	■	Relative	Gelu	□			4	60	4096	68	32	0
Falcon 2 11B	openai.com/research/falcon-2	2024	16384	16384	Serial	■	Relative	Gelu	□	2.5x		3.608	80	8192	102	64	0
Qwen 2 (72B) - same for 2.5	qwenlm.github.io	2024	250000	250000	Serial	■	Relative	Gelu	□			4	96	16432	192	96	0
NeMo-4 (440B)	nvidia.com/papers/p17603	2024	250000	250000	Serial	■	Relative	Gelu	□			4	96	4096	128	32	0
Qwen 2 (72B)	qwenlm.github.io	2024	250000	250000	Serial	■	Relative	Gelu	□			4	96	4096	128	32	0
Qwen 2 (72B)	qwenlm.github.io	2024	250000	250000	Serial	■	Relative	Gelu	□			4	96	4096	128	32	0
Qwen 2 (72B)	qwenlm.github.io	2024	250000	250000	Serial	■	Relative	Gelu	□			4	96	4096	128	32	0
Qwen 2 (72B)	qwenlm.github.io	2024	250000	250000	Serial	■	Relative	Gelu	□			4	96	4096	128	32	0
Qwen 2 (72B)	qwenlm.github.io	2024	250000	250000	Serial	■	Relative	Gelu	□			4	96	4096	128	32	0
Qwen 2 (72B)	qwenlm.github.io	2024	250000	250000	Serial	■	Relative	Gelu	□			4	96	4096	128	32	0
Qwen 2 (72B)	qwenlm.github.io	2024	250000	250000	Serial	■	Relative	Gelu	□			4	96	4096	128	32	0
Qwen 2 (72B)	qwenlm.github.io	2024	250000	250000	Serial	■	Relative	Gelu	□			4	96	4096	128	32	0
Qwen 2 (72B)	qwenlm.github.io	2024	250000	250000	Serial	■	Relative	Gelu	□			4	96	4096	128	32	0
Qwen 2 (72B)	qwenlm.github.io	2024	250000	250000	Serial	■	Relative	Gelu	□			4	96	4096	128	32	0
Qwen 2 (72B)	qwenlm.github.io	2024	250000	250000	Serial	■	Relative	Gelu	□			4	96	4096	128	32	0
Qwen 2 (72B)	qwenlm.github.io	2024	250000	250000	Serial	■	Relative	Gelu	□			4	96	4096	128	32	0
Qwen 2 (72B)	qwenlm.github.io	2024	250000	250000	Serial	■	Relative	Gelu	□			4	96	4096	128	32	0
Qwen 2 (72B)	qwenlm.github.io	2024	250000	250000	Serial	■	Relative	Gelu	□			4	96	4096	128	32	0
Qwen 2 (72B)	qwenlm.github.io	2024	250000	250000	Serial	■	Relative	Gelu	□			4	96	4096	128	32	0
Qwen 2 (72B)	qwenlm.github.io	2024	250000	250000	Serial	■	Relative	Gelu	□			4	96	4096	128	32	0
Qwen 2 (72B)	qwenlm.github.io	2024	250000	250000	Serial	■	Relative	Gelu	□			4	96	4096	128	32	0
Qwen 2 (72B)	qwenlm.github.io	2024	250000	250000	Serial	■	Relative	Gelu	□			4	96	4096	128	32	0
Qwen 2 (72B)	qwenlm.github.io	2024	250000	250000	Serial	■	Relative	Gelu	□			4	96	4096	128	32	0
Qwen 2 (72B)	qwenlm.github.io	2024	250000	250000	Serial	■	Relative	Gelu	□			4	96	4096	128	32	0
Qwen 2 (72B)	qwenlm.github.io	2024	250000	250000	Serial	■	Relative	Gelu	□			4	96	4096	128	32	0
Qwen 2 (72B)	qwenlm.github.io	2024	250000	250000	Serial	■	Relative	Gelu	□			4	96	4096	128	32	0
Qwen 2 (72B)	qwenlm.github.io	2024	250000	250000	Serial	■	Relative	Gelu	□			4	96	4096	128	32	0
Qwen 2 (72B)	qwenlm.github.io	2024	250000	250000	Serial	■	Relative	Gelu	□			4	96	4096	128	32	0
Qwen 2 (72B)	qwenlm.github.io	2024	250000	250000	Serial	■	Relative	Gelu	□			4	96	4096	128	32	0
Qwen 2 (72B)	qwenlm.github.io	2024	250000	250000	Serial	■	Relative	Gelu	□			4	96	4096	128	32	0
Qwen 2 (72B)	qwenlm.github.io	2024	250000	250000	Serial	■	Relative	Gelu	□			4	96	4096	128	32	0
Qwen 2 (72B)	qwenlm.github.io	2024	250000	250000	Serial	■	Relative	Gelu	□			4	96	4096	128	32	0
Qwen 2 (72B)	qwenlm.github.io	2024	250000	250000	Serial	■	Relative	Gelu	□			4	96	4096	128	32	0
Qwen 2 (72B)	qwenlm.github.io	2024	250000	250000	Serial	■	Relative	Gelu	□			4	96	4096	128	32	0
Qwen 2 (72B)	qwenlm.github.io	2024	250000	250000	Serial	■	Relative	Gelu	□			4	96	4096	128	32	0
Qwen 2 (72B)	qwenlm.github.io	2024	250000	250000	Serial	■	Relative	Gelu	□			4	96	4096	128	32	0
Qwen 2 (72B)	qwenlm.github.io	2024	250000	250000	Serial	■	Relative	Gelu	□			4	96	4096	128	32	0
Qwen 2 (72B)	qwenlm.github.io	2024	250000	250000	Serial	■	Relative	Gelu	□			4	96	4096	128	32	0
Qwen 2 (72B)	qwenlm.github.io	2024	250000	250000	Serial	■	Relative	Gelu	□			4	96	4096	128	32	0
Qwen 2 (72B)	qwenlm.github.io	2024	250000	250000	Serial	■	Relative	Gelu	□			4	96	4096	128	32	0
Qwen 2 (72B)	qwenlm.github.io	2024	250000	250000	Serial	■	Relative	Gelu	□			4	96	4096	128	32	0
Qwen 2 (72B)	qwenlm.github.io	2024	250000	250000	Serial	■	Relative	Gelu	□			4	96	4096	128	32	0
Qwen 2 (72B)	qwenlm.github.io	2024	250000	250000	Serial	■	Relative	Gelu	□			4	96	4096	128	32	0
Qwen 2 (72B)	qwenlm.github.io	2024	250000	250000	Serial	■	Relative	Gelu	□			4	96	4096	128	32	0
Qwen 2 (72B)	qwenlm.github.io	2024	250000	250000	Serial	■	Relative	Gelu	□			4	96	4096	128	32	0
Qwen 2 (72B)	qwenlm.github.io	2024	250000	250000	Serial	■	Relative	Gelu	□			4	96	4096	128	32	0
Qwen 2 (72B)	qwenlm.github.io	2024	250000	250000	Serial	■	Relative	Gelu	□			4	96	4096	128	32	0
Qwen 2 (72B)	qwenlm.github.io	2024	250000	250000	Serial	■	Relative	Gelu	□			4	96	4096	128	32	0
Qwen 2 (72B)	qwenlm.github.io	2024	250000	250000	Serial	■	Relative	Gelu	□			4	96	4096	128	32	0
Qwen 2 (72B)	qwenlm.github.io	2024	250000	250000	Serial	■	Relative	Gelu	□			4	96	4096	128	32	0
Qwen 2 (72B)	qwenlm.github.io	2024	250000	250000	Serial	■	Relative	Gelu	□			4	96	4096	128	32	0
Qwen 2 (72B)	qwenlm.github.io	2024	250000	250000	Serial	■	Relative	Gelu	□			4	96	4096	128	32	0
Qwen 2 (72B)	qwenlm.github.io	2024	250000	250000	Serial	■	Relative	Gelu	□			4	96	4096	128	32	0
Qwen 2 (72B)	qwenlm.github.io	2024	250000	250000	Serial	■	Relative	Gelu	□			4	96	4096	128	32	0
Qwen 2 (72B)	qwenlm.github.io	2024	250000	250000	Serial	■	Relative	Gelu	□			4	96	4096	128	32	0
Qwen 2 (72B)	qwenlm.github.io	2024	250000	250000	Serial	■											